EqTrace: Inspectable Contracts between Papers, Code, and Execution

Shiqi Wang
HKUST(GZ), Urban Governance and Design

2026-09-05

Abstract

A research method can be described correctly while its code follows an unreported path or its reported artifacts come from an earlier run. EqTrace is a lightweight prototype for inspecting these relationships across a paper and an engineering project. A manifest links codebase roots, datasets, hierarchical semantic blocks, virtual interfaces, and declared runs. Static scans and recorded execution events populate interactive flowcharts and dataset tables. At selected leaves, LaTeX equations or straight-line pseudocode and independently written Python functions lower to a shared ordered representation. Separate checks establish structural agreement, conditional real equivalence, and sampled execution agreement. A synthetic sequence example spans 36 Python files across 2 codebases; it exercises preprocessing, a frozen attention encoder, regression-head training, adaptation, and evaluation. All 4 declared array-shape contracts matched observed calls. A scalar fault corpus matched 18 of 18 authored expectations, while a midpoint-only check accepted 4 invalid cases. The repository also checks its own discrepancy kernels and scans its own architecture. These results demonstrate an auditable engineering workflow. Whole-block meaning remains declared, and mathematical proof is restricted to selected scalar contracts.

1 Introduction

Research implementations distribute a method across mathematical statements, data preparation, model components, optimization loops, and artifact writers. A plausible final result does not establish that each component followed the paper. A normalization stabilizer can disappear from a formula's implementation; a fallback can bypass a requested method; a checkpoint can remain from a previous run. Each failure requires evidence at a different scale.

The problem addressed here is how to make those relationships inspectable before merging a research change. A useful check must connect statements to current source locations and distinguish intended structure from observed execution. For a multi-file project, that connection also requires intermediate entities: a semantic block can comprise files from several codebases, and an interface can describe an in-memory array with no dedicated source file.

Mathematical document systems already provide executable notation. I Heart LA generates code from a linear-algebra language (Li et al., 2021). HeartDown embeds executable formulas and dynamic figures in scientific documents (Li et al., 2022). EqTrace addresses a complementary review workflow in which paper and code remain independently authored sources. The author declares their bindings, and deterministic checks retain disagreements rather than automatically reconciling the sources.

EqTrace combines an engineering evidence graph with local executable equation contracts. The graph organizes source observations, dataset statistics, declared block meaning, and bounded run traces. Selected equation leaves receive stricter parsing, real-arithmetic checks, and execution comparisons. Both layers expose their evidence through interactive views and machine-readable reports. The evaluation comprises a synthetic multi-file pipeline, an authored scalar fault corpus, and self-application. The resulting contribution is a reproducible prototype and an explicit evidence contract; comparative effectiveness on independent research repositories remains unmeasured.

2 Related work

Executable mathematical languages establish a direct path from notation to implementation. I Heart LA defines a compilable language resembling conventional linear algebra and targets LaTeX, Python, C++, and MATLAB (Li et al., 2021). HeartDown extends this direction to scientific documents, executable formulas, and dynamic figures (Li et al., 2022). These systems motivate a shared semantic representation. EqTrace uses a smaller scalar language to compare existing sources and retain disagreements as reviewable artifacts. We make no novelty claim for equation-to-code generation itself.

Proof assistants provide a stronger certificate boundary than this prototype. Lean is a programming language and theorem prover supporting formalized mathematics and verification (de Moura & Ullrich, 2021). Prove2Me organizes collaborative formalization missions whose results are checked in Lean (Chen et al., 2026). Its platform documents kernel checking, axiom restrictions, and pinned environments (Prove2Me, 2026). EqTrace borrows the emphasis on explicit trust and reproducibility. Its Z3 results depend on the implemented translation and solver, and are reported separately from runtime observations.

Source visualization answers a different class of review questions. The code2flow project constructs approximate call graphs for dynamic languages (Rogowski, n.d.). Such graphs help locate relationships between functions. EqTrace instead represents the scalar operations of a bound computation, merges equal subexpressions, and attaches source locations and verification records. Local inspection of LLM visualization projects informed the linked-node interface and the separation between semantic data and rendering; their code was not incorporated.

3 Method

3.1 The engineering graph separates declared meaning from observations

An architecture manifest names explicit codebase roots, local datasets, semantic blocks, virtual entities, and runs. Each block declares its meaning, file-membership selectors, and input/output entity identifiers. A parent relation supports nested blocks. The same block can include files from several roots, and named flows select views of the same entities for preprocessing, training, adaptation, or model inspection. Invalid memberships, hierarchy cycles, and dangling interfaces block the architecture check.

The scanner parses Python files into an abstract syntax tree and collects symbols, imports, candidate call edges, and review findings such as exception handlers. Import and call matching can cross the declared codebase roots. Dynamic targets remain unresolved. These observations support navigation; they do not establish a complete call graph or infer the scientific meaning of a block.

Dataset inspection computes a full-file hash and streams local comma-separated values (CSV) or JSON Lines (JSONL) records. It records observed types, missing values, and numeric ranges, then emits a LaTeX table from those statistics. A row limit produces an explicitly partial observation, and a required complete scan cannot pass in that state. Row contents are excluded from the exported summary. Column names and ranges can still carry sensitive information and remain subject to the author's release boundary.

3.2 Explicit execution binds blocks to current run evidence

The trace command executes a declared Python entry point in a child process. It records main-thread function calls and executed lines within selected source roots. For NumPy arrays, argument and return observations retain shapes and data types. Declared shape axes can contain fixed dimensions or symbolic dimensions; a repeated symbol enforces equality within one observation. Shape matches are observations of exercised calls, with no all-input guarantee.

A run receipt binds the command to current source, dataset, checker, runtime, and output hashes. Declared outputs must be opened for writing during the run and exist afterwards. Merely finding an old output is insufficient. Rechecking rejects changed sources, datasets, artifacts, or recorded dependencies.

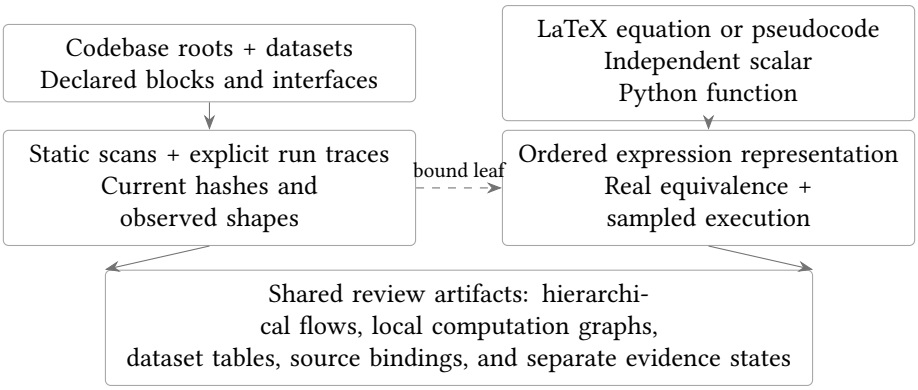

Figure 1: Engineering observations and local equation checks meet in a shared inspection view. Block meaning remains declared, while proof status belongs to an explicitly bound scalar leaf.

A block is marked as having observed calls only if a function in a member file was called; module import alone does not satisfy this condition.

The trace boundary covers selected Python files and the main thread. Native kernels, other threads, subprocesses, and the internal behavior of external dependencies remain outside it. File-open observations establish that a write path was exercised; they do not prove that an output was fully regenerated or scientifically valid. The pipeline command runs with the caller's ordinary permissions, so tracing requires trusted project code. Static scanning does not execute source files.

3.3 An equation contract binds independent sources

An equation contract identifies a labeled paper display, a Python function, and an ordered list of scalar inputs. It also declares a closed real interval for every input, optional nonzero assumptions, and a merge policy. Authors retain both source files. EqTrace reports disagreements and exports candidate translations without modifying either source.

The paper frontend reads explicitly selected LaTeX files. Every labeled equation or algorithm in those files must have a contract or a reasoned exclusion. This inventory measures coverage within the selected files. It does not establish coverage of an entire manuscript, its transitive inputs, or unlabeled mathematical prose.

Both frontends lower expressions into an ordered intermediate representation (IR), within the architecture in Figure 1. Each node contains an operation, ordered operands, an optional rational constant or exponent, and a source location. The accepted arithmetic includes addition, subtraction, multiplication, division, negation, and small literal integer powers. Square root, exponential, and logarithm can be translated and sampled, but the initial solver backend rejects them as unsupported.

3.4 Parsing rejects undeclared execution paths

The LaTeX parser consumes the complete expression and checks every symbol against the input declaration. Single letters, numeric subscripts, selected Greek commands, and explicit multi-letter identifiers have defined meanings. Unknown commands, incomplete expressions, and unsupported display environments produce a blocking diagnostic. This restricted grammar avoids treating LaTeX typography as an unambiguous programming language; general-purpose LaTeX parsing also has documented ambiguity and incomplete-parse risks (SymPy Development Team, n.d.).

The pseudocode frontend accepts a straight-line subset of the `algorithmic` environment. It requires an ordered input declaration, permits single-assignment statements, and requires one final return. Assignment expressions use the same scalar grammar. Branches, loops, and statements following a return are rejected. The Python frontend applies the corresponding restrictions to the abstract syntax tree (AST). It rejects decorators, dynamic calls, exception handlers, input reassignment, and unused computations.

Execution uses the validated target function extracted from its source AST. EqTrace does not import the project module or call an alternate implementation when extraction fails. Module headers with unmodeled evaluation are rejected. The resulting evidence applies to the extracted function with ordinary scalar arguments. External callers, module initialization, dependency dispatch, and production tensor kernels remain outside this boundary.

3.5 Equivalence is conditional on a nonempty, defined domain

The proof backend translates rational expressions into Z3 real arithmetic. It first checks whether the declared assumptions have a model. It then asks whether any division or negative power can be undefined. Only after these checks succeed does it search for inputs where the two outputs differ. An unsatisfiable final query produces the status PROVED_REAL; a satisfying model becomes an inspectable counterexample.

Explicit totality checks are necessary because division by zero is underspecified in Z3's arithmetic semantics (Z3 Contributors, n.d.). Adding nonzero guards only to the final equality query could hide a missing domain assumption. EqTrace instead rejects an admitted input that makes either expression undefined. Solver timeouts and unknown results retain unresolved status, with a timeout applied to each query.

The result is solver evidence under the implemented translation and declared real semantics. Replayable SMT-LIB queries and the solver version accompany it. EqTrace does not produce a Lean proof term or independently check an exported proof certificate. Correctness of the parsers, translation, solver, and expression semantics remains part of the trusted implementation.

3.6 Execution checks remain separate from real equivalence

Sampled execution compares the current source function with an equation interpreter. Boundary and coordinate probes precede seeded random probes within the declared intervals. Duplicate floating-point inputs are removed. The run fails if it cannot produce the requested number of distinct admissible samples. Rational expressions use exact fractions for the reference calculation on the sampled binary input values. Elementary functions explicitly use the standard mathematical library.

The checker uses a normalized squared discrepancy to compare finite scalar results. For an actual value a and a reference value b, the squared-error kernel is

$$e = (a - b)^2 \tag{1}$$

The normalized kernel is

$$r = \frac{(a - b)^2}{1 + b^2} \tag{2}$$

A sample passes when r is no greater than the square of the declared tolerance. The discrepancy itself is evaluated using fractions to prevent overflow or cancellation inside the comparison metric. An undefined, nonscalar, or nonfinite result fails the sample.

The normalized kernel is also specified as executable pseudocode in Algorithm 1. EqTrace checks both paper forms against the same implementation used by the execution harness. The squared kernel has a separate contract. These checks demonstrate self-application to small arithmetic kernels; they do not verify the checker that performs the checks.

3.7 The merge policy preserves distinct evidence levels

For an equation contract, a strict merge pass requires a resolved source binding, a real-equivalence result, and a successful execution comparison. An ordered policy additionally requires identical ordered expressions. An algebraic policy allows different computation graphs when the remaining checks pass. A user can request sampling without the solver, but its status is SAMPLED_ONLY and its exit code cannot satisfy the strict merge check.

Algorithm 1 Normalized squared discrepancy

Require: a, b
Ensure: r
$d \leftarrow a - b$
$v \leftarrow d \cdot d$
$w \leftarrow 1 + b \cdot b$
return $\frac{v}{w}$

A receipt records source hashes, checker hashes, configuration, environment, solver queries, and per-sample results. Generated code, LaTeX, pseudocode, JSON graphs, Mermaid diagrams, and SVGs retain their own artifact hashes. Receipt verification rejects changed sources, changed artifacts, and changed checker or runtime versions. These hashes support accidental-change detection and freshness checks. They provide no signature-based authenticity guarantee.

3.8 A shared view preserves the evidence boundary

The engineering report distinguishes a successful static scan from a successful audit of required runs. A selected scalar contract can attach to a block only when its implementation belongs to that block. Its local proof and execution status remain separate from the block's declared meaning. In particular, proving a squared-error leaf does not establish the correctness of the surrounding evaluation pipeline.

Offline HTML workbenches render the report without remote assets. The engineering view provides named flows, block-to-file inspection, cross-codebase dependencies, dataset tables, shape observations, and run evidence. The equation view provides a merged operation graph, source locations, solver witnesses, and sampled values. JSON exports expose the same evidence to automated reviewers. The optional live equation editor uses a loopback-only service to check temporary source pairs; it does not launch general pipeline runs.

4 Prototype evaluation

4.1 A multi-file sequence pipeline exercises the engineering layer

The engineering example comprises 36 Python source files across 2 codebase roots. Its manifest defines 7 semantic blocks and 3 named flows. Membership includes shared numerical and serialization utilities in the second root. The example uses 384 synthetic sequence records; their 9 columns are summarized directly from the scanned file in Table 1. The deterministic generator and its seed are included in the release.

The pipeline prepares arrays, applies a frozen single-block attention encoder, trains a linear regression head, adapts that head, and evaluates held-out records. The attention module performs query, key, and value projections followed by scaled dot products and softmax. The parent block adds residual paths and a feed-forward transformation. Encoder parameters remain fixed throughout. Only the regression head is optimized, so this example supplies no evidence about full Transformer training or model quality on an external task.

The retained run produced a prepared dataset and two checkpoints. It also wrote predictions and loss records. Main-thread events covered 36 selected source files, and all 4 declared array-shape contracts matched observed calls. One scalar squared-error contract was bound to the prediction writer's block and passed separately. The architecture audit accepted the current run and its artifacts.

The recorded head-training loss changed from 0.058081 to 0.001825 over 180 updates. Adaptation loss changed from 0.003019 to 0.002592 over 80 updates. The held-out mean squared error (MSE) was 0.001561 across 64 records. These values confirm that optimization and evaluation produced inspectable numerical artifacts; they are not a comparison against another learning method.

Field	Observed type	Missing	Numeric range
sample_id	integer	0/384	0 to 383
split	string	0/384	–
x0	number	0/384	-0.978969 to 1.23217
x1	number	0/384	-0.995538 to 1.19791
x2	number	0/384	-0.988457 to 1.18068
x3	number	0/384	-0.99404 to 1.22788
x4	number	0/384	-0.99235 to 1.19192
x5	number	0/384	-0.999572 to 1.12618
target	number	0/384	-0.46891 to 0.863679

Table 1: The dataset scan found a complete synthetic table and generated its field summary directly from observed records. Ranges describe this fixture and carry no inferred domain meaning.

4.2 An authored corpus exercises the declared failure boundary

The evaluation comprised 18 synthetic cases specified before the reported run. 4 represented valid implementations, including an algebraic rewrite and a pseudocode source. The remaining cases exercised formula drift, missing implementations, unsupported control paths, domain failures, and floating-point divergence. Each case retained its exact paper source, implementation, manifest, and generated receipt. Table 2 was generated from those run records.

Each runnable case requested 64 distinct inputs with seed 1729 and tolerance 10^{-10}. Proof queries had a 3000 millisecond timeout per query. The released run recorded CPython 3.13.12 and Z3 4.16.0 on an ARM64 macOS host. The total number of executed samples was 640. Cases rejected before translation generated no execution evidence. The corpus is a regression suite authored alongside the tool, with no statistical sampling model or real-world prevalence estimate.

Authored case	Target	Observed	Real check
exact square	Pass	Pass	Proved
equivalent expansion	Pass	Pass	Proved
explicit epsilon	Pass	Pass	Proved
pseudocode square	Pass	Pass	Proved
constant drift	Fail	Fail	Counterexample
missing cross term	Fail	Fail	Counterexample
ignored epsilon	Fail	Fail	Counterexample
identity for square	Fail	Fail	Counterexample
hidden fallback	Blocked	Blocked	Not run
commented plan	Blocked	Blocked	Not run
discarded computation	Blocked	Blocked	Not run
unknown equation	Blocked	Blocked	Not run
missing implementation	Blocked	Blocked	Not run
division domain hole	Fail	Fail	Domain error
vacuous domain	Fail	Fail	Empty domain
float cancellation	Fail	Fail	Proved
unmodeled call	Blocked	Blocked	Not run
pseudocode branch	Blocked	Blocked	Not run

Table 2: The prototype matched all authored classifications in the released synthetic corpus. Blocked cases lack accepted executable semantics; they are counted separately from demonstrated inequivalence.

4.3 The evidence layers produced different decisions

EqTrace matched 18 of 18 authored expectations. It accepted 4 valid cases and produced no strict pass among the 14 invalid or unsupported cases. These counts describe the provided fixtures. They do not

establish sensitivity or specificity on independent research repositories.

Two internal ablations exposed why one check is insufficient. The midpoint-only ablation accepted 4 invalid cases that the combined check rejected. The ordered-structure ablation rejected 1 valid algebraic rewrite. Both ablations reused EqTrace's strict frontends; unavailable results were not credited as detected bugs. They are component comparisons within this prototype, rather than evaluations of external competing systems.

The floating-point cancellation case passed the real-equivalence query and failed the execution comparison. Its implementation added and subtracted 10^{16} around a scalar input. The resulting real expression equals that input, while finite-precision evaluation lost information for tested values. This case demonstrates why PROVED_REAL and execution success occupy separate fields in the receipt.

4.4 Self-application checks the arithmetic used by the checker

The self-application manifest linked Equations 1 and 2, together with Algorithm 1, to functions in the released package. All 3 contracts passed their real-equivalence and sampled execution checks. The normalized discrepancy function is called by the runtime comparison harness. This closes a small paper-to-code-to-evidence cycle within the repository.

The self-application result has a circular trust limitation. The same implementation parses the paper statement, lowers the code, and judges their relationship. A shared translation bug could survive the check. Independent proof-certificate checking, a second frontend implementation, and external fixtures would provide different evidence, and remain future work.

4.5 Regression and browser checks exercise failure handling

The released local test run passed 80 tests. Engineering regression cases changed source files, datasets, and outputs after a run; supplied a pre-existing unwritten artifact; imported a module without calling its block function; and declared an incorrect array dimension. Each case required a blocking result. The suite also exercised scalar parsing, domain failures, numerical disagreement, stale receipts, and constrained live comparisons.

Browser checks exercised named flows, cross-codebase block inspection, dataset-to-LaTeX display, observed array shapes, and attached scalar proof evidence. Separate checks exercised equation rendering, merged graph inspection, live mismatch detection, exports, and a narrow viewport. These checks establish selected user-interface behavior on Chromium. They do not establish accessibility conformance or behavior in every browser.

The package also scans its own Python roots into semantic blocks and binds its three local self-contracts to the checker block. This view remains a static architecture observation. The release does not label the entire checker as dynamically covered or mathematically verified.

5 Discussion and limitations

EqTrace provides a reviewable boundary between a method's intended organization and the evidence collected from its implementation. Authors can navigate from a declared stage to current source files, inspect which calls were observed, and examine a local equation disagreement. Unsupported scalar paths remain blocking. Complex project paths can still be scanned and traced, with unresolved behavior visible at the engineering layer.

The principal semantic limit is that whole-block meaning and dataflow are authored declarations. Static dependencies and bounded run events cannot prove that a preprocessing stage has the stated meaning or that training implements every equation in a paper. The current mathematical layer accepts only a restricted scalar language. Tensor operators, iteration, state, automatic differentiation, and optimized kernels require additional semantics before they can receive comparable proof claims.

Real and executable semantics also differ by construction. Solver checks use mathematical reals and exact decimal constants. Execution observes CPython operations and binary inputs. Sampling

can expose numerical discrepancies but cannot establish a global floating-point error bound. Interval analysis, floating-point solver encodings, or validated numerics could strengthen selected operators.

The evidence records support freshness checks for the observed environment. They are not authenticated attestations, and a malicious process could manipulate its tracing environment. Recorded dependency versions cover selected observed numerical libraries, not the complete operating system or accelerator stack. Matching output hashes and write events establish a narrower fact than a full reproducible build. The author must define which sources, data, dependencies, and outputs are material to the experiment.

The evaluation is an authored regression suite and one synthetic engineering example. This design exercises implemented guarantees and rejection states. It cannot establish detection effectiveness or practical overhead on unseen research repositories. An independent study should include valid alternative implementations, realistic tolerances, ambiguous block boundaries, and disagreements labeled without EqTrace's output.

The workbench can assist human or AI review through common structured evidence. Scientific assumptions, chosen domains, and empirical conclusions remain author and reviewer judgments. A future Lean backend could provide independently checked expression semantics or translations. That extension would still need an explicit connection to the code actually executed.

6 Conclusion

EqTrace connects paper statements to code through hierarchical engineering contracts and stricter local equation checks. The prototype scanned a multi-codebase sequence pipeline, observed its declared run, and exported data summaries and interactive flows. Its scalar corpus and self-application demonstrated the intended acceptance and rejection behavior within a restricted language. The reusable result is a local review and merge-check tool whose evidence states distinguish declarations, observations, sampled agreement, and conditional real proof.

Code, data, and manuscript availability

The source, MIT license, synthetic corpus, execution records, and this LaTeX manuscript are released together at https://github.com/Mappedinfo/eqtrace. The repository includes commands for regeneration and continuous integration. No private research data, model weights, or external API credentials are required.

Development disclosure

The author proposed the paper–code consistency problem and requested this prototype. An AI coding assistant assisted implementation, test execution, and manuscript drafting. Reported classifications come from retained tool executions. This document is a software technical report and has not undergone peer review.

A Abbreviations

Term	Meaning
AST	Abstract syntax tree
CSV	Comma-separated values
HTML	Hypertext Markup Language
IR	Intermediate representation
JSON	JavaScript Object Notation
JSONL	JSON Lines
MSE	Mean squared error
SMT	Satisfiability modulo theories
SVG	Scalable Vector Graphics

References

Chen, S., Marwaha, K., Lu, X., Yuen, H., & Peng, T. (2026). Prove2Me: An open collaborative platform for scaling math formalization. https://doi.org/10.48550/arXiv.2608.28433

de Moura, L., & Ullrich, S. (2021). The Lean 4 theorem prover and programming language. *Automated Deduction – CADE 28*, 625–635. https://doi.org/10.1007/978-3-030-79876-5_37

Li, Y., Kamil, S., Jacobson, A., & Gingold, Y. (2021). I Heart LA: Compilable markdown for linear algebra. *ACM Transactions on Graphics*, *40*(6). https://iheartla.github.io/

Li, Y., Kamil, S., Jacobson, A., & Gingold, Y. (2022). HeartDown: Document processor for executable linear algebra papers. *SIGGRAPH Asia 2022 Conference Papers*. https://doi.org/10.1145/3550469.3555395

Prove2Me. (2026). *About Prove2Me*. Retrieved September 5, 2026, from https://prove2.me/about

Rogowski, S. (n.d.). *Code2flow: Pretty good call graphs for dynamic languages*. Retrieved September 5, 2026, from https://github.com/scottrogowski/code2flow

SymPy Development Team. (n.d.). *Parsing: SymPy documentation*. Retrieved September 5, 2026, from https://docs.sympy.org/latest/modules/parsing.html

Z3 Contributors. (n.d.). *Arithmetic: Online Z3 guide*. Retrieved September 5, 2026, from https://microsoft.github.io/z3guide/docs/theories/Arithmetic/